

OS LAB-5
DATE: 23/3/26

SUMANT SHRIDHAR
USN: 1BM24CS299
BATCH: 4K2

1) Rate Monotonic Scheduling

```
#include <stdio.h>
#define MAX 20

struct Task {
    int pid;
    int C;
    int P;
    int rem;
    int next_arrival;
};

int main() {

    int n, i, j, time, max_time;
    struct Task t[MAX];

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        t[i].pid = i + 1;
        printf("P%d Execution Time: ", i + 1);
        scanf("%d", &t[i].C);
        printf("P%d Period: ", i + 1);
        scanf("%d", &t[i].P);
    }

    printf("Enter total time: ");
    scanf("%d", &max_time);

    //sort priority wise
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (t[j].P > t[j + 1].P) {
                struct Task temp = t[j];
                t[j] = t[j + 1];
                t[j + 1] = temp;
            }
        }
    }

    //init
    for (i = 0; i < n; i++) {
        t[i].rem = 0;
        t[i].next_arrival = 0;
    }
}
```

```

printf("\nTime\tTask\n");

for (time = 0; time < max_time; time++) {

    //check arrival
    for (i = 0; i < n; i++) {
        if (time == t[i].next_arrival) {
            t[i].rem = t[i].C;
            t[i].next_arrival += t[i].P;
        }
    }

    //get highest priority
    int current = -1;
    for (i = 0; i < n; i++) {
        if (t[i].rem > 0) {
            current = i;
            break;
        }
    }

    if (current != -1) {
        printf("%d\tP%d\n", time, t[current].pid);
        t[current].rem--;
    } else {
        printf("%d\tIdle\n", time);
    }

    //check deadline
    for (i = 0; i < n; i++) {
        if (time > (t[i].next_arrival + t[i].P) && t[i].rem > 0) {
            printf("Deadline Miss for Task T%d at time %d\n",
                t[i].pid, time);
        }
    }

}

return 0;
}

```

• Enter number of tasks: 3
P1 Execution Time: 3
P1 Period: 20
P2 Execution Time: 2
P2 Period: 5
P3 Execution Time: 2
P3 Period: 10
Enter total time: 20

Time	Task
0	P2
1	P2
2	P3
3	P3
4	P1
5	P2
6	P2
7	P1
8	P1
9	Idle
10	P2
11	P2
12	P3
13	P3
14	Idle
15	P2
16	P2
17	Idle
18	Idle
19	Idle

Lab 5

classmate

Date
Page

23/3/26

Rate Monotonic Scheduling

```
#include <stdio.h>
```

```
#define MAX 20
```

```
struct Task {
```

```
    int pid;
```

```
    int C;
```

```
    int P;
```

```
    int rem;
```

```
    int next_arrival;
```

```
};
```

```
int main() {
```

```
    int n, i, j, time, max_time;
```

```
    struct Task t[MAX];
```

```
    printf("Enter no. of tasks: ");
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++) {
```

```
        t[i].pid = i + 1;
```

```
        printf("P/d Execution Time: ", i + 1);
```

```
        scanf("%d", &t[i].C);
```

```
        printf("P/d Period: ", i + 1);
```

```
        scanf("%d", &t[i].P);
```

```
    printf("Enter total time: ");
```

```
    scanf("%d", &max_time);
```

```
    // sort priority wise
```

```
    for (i = 0; i < n - 1; i++) {
```

```
        for (j = 0; j < n - i - 1; j++) {
```

```
            if (t[j].P > t[j + 1].P) {
```



```

    struct Task temp = t[j];
    t[j] = t[j+1];
    t[j+1] = temp;
}
}
// initialise
for(i=0; i<n; i++){
    t[i].rem = 0;
    t[i].next_arrival = 0;
}
printf("\n Time \t Task \n");
for(time = 0; time < max_time; time++){
    // check arrival
    for(i=0; i<n; i++){
        if(time == t[i].next_arrival){
            t[i].rem = t[i].C;
            t[i].next_arrival += t[i].P;
        }
    }
    // get highest priority
    int current = -1;
    for(i=0; i<n; i++){
        if(t[i].rem > 0){
            current = i;
            break;
        }
    }
    if(current != -1){
        printf("%d \t P %d \n", time, t[current].P);
        t[current].rem--;
    }
}

```



```

else {
    printf("/.d)\tIdle\n", time);
}

```

//check deadline

```

for (i = 0; i < n; i++) {
    if (time > (t[i].next_arrival - t[i].P)) {
        t[i].rem > 0) {

```

```

        printf("Deadline Missed for Task %.d at time %.d\n", t[i].pid, time);

```

```

        i++;
    }
}

```

```

return 0;

```

```

}
}

```

```

}

```

~~giving below fig~~

```

}
}
}
}
}

```

```

}

```

```

}

```

Output:

Enter no
P1 Exec
P1 Per
P2 Exec
P2 Per
P3 Exec
P3 Per
Enter t

Time

0

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

"time):

Output:

Enter no. of tasks: 3

P1 Execution Time: 3

P1 Period: 20

P2 Execution Time: 2

P2 Period: 5

P3 Execution Time: 2

P3 Period: 10

Enter total time: 20

Time	Task
0	P2
1	P2
2	P3
3	P3
4	P1
5	P2
6	P2
7	P1
8	P1
9	Idle
10	P2
11	P2
12	P3
13	P3
14	Idle
15	P2
16	P2
17	Idle
18	Idle
19	Idle

{
t-arrival +
22

ssed for
ime y.d ln",
time):